

MOVELINK

System-Dokumentation & Traceability-Report

Anforderungs- & Abdeckungsanalyse der MoveLink App

Status: Freigegeben

Sprache: Deutsch

Autoren: Erlind & AI Assistant Antigravity

Dokumentenversion: v1.1.0 (Scraped Live)

Veröffentlichungsdatum: 20. Juni 2026

1. Dokumente aus dem Projekt

In diesem Abschnitt werden die im Projekt gefundenen Markdown-Dateien im Volltext aufgeführt. Diese Dokumente bilden die Grundlage für die Anforderungen und Use Cases.

Datei: doc/Requirements.md

Funktionale Anforderungen

- **FA1:** Das System muss einen Reiter oder eine Navigationsmöglichkeit für Hardwaregeräte / Trainings / vergangene Trainingseinheiten anzeigen. (aus **UC-1**, **UC-2**, **UC-3**)
- **FA2:** Das System muss mir eine Liste von verfügbaren Hardwaregeräte und mit welchen Hardwaregeräten ich verbunden bin anzeigen. (aus **UC-1**)
- **FA3:** Das System muss einen verbindungsaufruf mit dem Hardwaregerät herstellen können. (aus **UC-1**)
- **FA4:** Das System muss eine Detailansicht für ein ausgewähltes Training anzeigen. (aus **UC-2**)
- **FA5:** Das System muss Bewegungsdatenströme empfangen und verarbeiten können. (aus **UC-2**)
- **FA6:** Das System muss die empfangenen Bewegungen in Echtzeit visualisieren. (aus **UC-2**)
- **FA7:** Das System muss historische Bewegungsdaten grafisch anzeigen können. (aus **UC-3**)

Nicht-funktionale Anforderungen – Muss-Kriterien

- **NF1** – Latenz: Die E2E-Latenz vom Sensor bis zur Darstellung muss ≤ 100 ms sein.
- **NF2** – Zuverlässigkeit: Bei einem Verbindungsabbruch muss die App den Nutzer benachrichtigen und automatisch Reconnect-Versuche starten.
- **NF3** – Usability: Das Pairing darf maximal zwei Nutzerinteraktionen erfordern.

Rahmenbedingungen

- **R1:** Die Applikationen muss auf android devices laufen.

Datei: doc/UseCases.md

UC-1: Trainingsgerät verbinden

Akteur: Trainierender Vorbedingung: Trainingsgerät eingeschaltet

Beschreibung: Als Trainierender möchte ich mein Trainingsgerät mit der App verbinden können, um Trainingsdaten erfassen zu können.

1. Eingabe: Ich als trainierender öffne die App, Ausgabe: Ich sehe eine Möglichkeit/Reiter Hardwaregeräte anzuzeigen.

2. Eingabe: Ich als trainierender klicke auf den Reiter Hardwaregeräte, Ausgabe: Ich sehe eine Liste der verfügbaren Hardwaregeräte und mit welchen Hardwaregeräten ich verbunden bin.

3. Eingabe: Ich als trainierender klicke auf ein Hardwaregerät in der Liste auf verbinden, Ausgabe: Ich sehe eine Detailansicht des ausgewählten Hardwaregeräts und dass dies mit der App verbunden ist.

UC-2: Echtzeit-Training überwachen

Akteur: Trainierender Vorbedingung: Trainingsgerät verbunden

Beschreibung: Ich als Trainierender möchte mein Training in Echtzeit überwachen können, um direkt Feedback zu meiner Ausführung zu erhalten.

1. Eingabe: Ich als trainierender öffne die App, Ausgabe: Ich sehe eine Möglichkeit/Reiter mein Training zu starten.

2. Eingabe: Ich als trainierender klicke auf den Reiter "Training starten", Ausgabe: Ich sehe eine Detailansicht des ausgewählten Trainings und die Möglichkeit mein Training zu starten.

3. Eingabe: Ich als trainierender drücke den Start Button in der Detailansicht des Trainings, Ausgabe: Die App demonstriert mir Bewegungen, welche nachzumachen sind.

4. Eingabe: Ich als trainierender mache die Bewegung nach, Ausgabe: Die App visualisiert die Bewegung in Echtzeit und vergleicht diese mit der Demonstration. Gibt positives Feedback, wenn die Bewegung korrekt ausgeführt wurde.

UC-3: Trainingsdaten einsehen

Akteur: Trainierender Vorbedingung: Vergangene Trainingseinheit vorhanden

Beschreibung: Ich als Trainierender möchte vergangene Trainingseinheiten einsehen können, um meine Fortschritte zu verfolgen.

1. Eingabe: Ich als trainierender öffne die App, Ausgabe: Ich sehe eine Möglichkeit/Reiter

vergangene Trainingseinheiten anzuzeigen.

2. Eingabe: Ich als trainierender klicke auf den Reiter "Trainingseinheiten", Ausgabe: Ich sehe eine Liste der vergangenen Trainingseinheiten an.

3. Eingabe: Ich wähle eine Trainingseinheit aus der Liste aus, Ausgabe: Die App zeigt die historischen Bewegungsdaten grafisch an.

Datei: app/components/ProfileCard/architecture.md

Profil-Karte (ProfileCard)

Diese Komponente zeigt die Profildetails des angemeldeten Benutzers an.

Datenfluss

```

flowchart LR
    JWT[Authentifizierungs-Token] -->|1. User-ID auslesen| Controller[ProfileController]
    Controller -->|2. Query| DB[(Datenbank)]
    DB -->|3. Rohdaten| Controller
    Controller -->|4. Bereinigtes Profil DTO| Client[ProfileCard UI]
```

Datei: app/components/architecture.md

App Komponenten

Diese Verzeichnis enthält die UI-Komponenten der Mobile-/Web-Anwendung (z.B. Visualisierungen, Charts, Navigation).

Datenfluss in UI-Komponenten

Die Komponenten erhalten Daten über Props oder den globalen Store und rendern diese reaktiv.

```

flowchart LR
    Store[Globaler Store] -->|Reaktive Updates| SensorCard[SensorCard / LiveChart]
    SensorCard -->|Klick / Aktion| Actions[Store Actions]
    Actions -->|Dispatch| Store
```

- **LiveChart:** Rendert eintreffende Datenpunkte in Echtzeit.
- **SensorCard:** Zeigt den aktuellen Status und Verbindungszustand von Bluetooth-Geräten.

Datei: doc/firstnotes.md

Mikrocontroller <--- Datenpakete ---> App

Backend <- API -> Frontend

(Backend?)

Problemstellung:

Darstellung von Trainingsdatensätzen zur Bewegungsanalyse.

Projektebene

Stakeholder:

- Trainierenden (Fokus)
- Entwickler
- Dienstleister
- Hochschule

Anforderungen an die App:

2 Zustände:

1 Zustand (Fokus) aktives Training:

- Intention: visuelle Darstellung des Trainingsfortschritts in Echtzeit
- FA: In Echtzeit eine Bewertung abliefern
- FA: Darstellung der Ausführung
- NFA: Eine eigenständige Applikation
- NFA: Das Darstellen der darf Höchstens 1 Sekunden dauern
- Rahmenbedingung: Auf Android, Mac und Windows funktionsfähig
- Technologien: Flutter & MAUI

Ich habe einen XIOA nRF52840 Gerät. Ich möchte die Daten in Echtzeit in einem Frontend anzeigen z.B. React. Wie sinnvoll und komplex wäre es einen Server dazwischen zu packen?

Datei: [embedded/src/architecture.md](#)

Embedded Sensor-Firmware

Die Firmware liest Sensorwerte aus und überträgt diese über Bluetooth Low Energy (BLE) an die App.

Datenfluss Firmware

flowchart TD

```

    Sensor[MPU6050 Beschleunigungssensor] -->|I2C Rohdaten| Arduino[Arduino / ESP32 Controller]
    Arduino -->|Signalfilterung & Skalierung| BLE[Bluetooth Low Energy Characteristic]
    BLE -->|Notifikationen / Byte-Array| App[Mobile App]
  
```

- **Sensordatenerfassung:** Erfolgt in einer festen Frequenz (z.B. 50Hz).
- **Filterung:** Tiefpassfilter zur Rauschminderung auf dem Mikrocontroller.
- **BLE-Transfer:** Effiziente Übertragung als binäres Datenpaket.

2. Requirements vs. Use Cases Matrix

Die folgende Matrix veranschaulicht die Beziehungen zwischen den definierten funktionalen Anforderungen (FA) / nicht-funktionalen Anforderungen (NF) / Rahmenbedingungen (R) und den Use Cases (UC).

Anforderung / ID	UC-1	UC-2	UC-3
FA1: Das System muss einen Reiter oder eine Naviga...	X	X	X
FA2: Das System muss mir eine Liste von verfügbare...	X	-	-
FA3: Das System muss einen Verbindungsaufbau mit d...	X	-	-
FA4: Das System muss eine Detailansicht für ein au...	-	X	-
FA5: Das System muss Bewegungsdatenströme empfange...	-	X	-
FA6: Das System muss die empfangenen Bewegungen in...	-	X	-
FA7: Das System muss historische Bewegungsdaten gr...	-	-	X
NF1: Latenz: Die E2E-Latenz vom Sensor bis zur Dar...	-	-	-
NF2: Zuverlässigkeit: Bei einem Verbindungsabbruch...	-	-	-
NF3: Usability: Das Pairing darf maximal zwei Nutz...	-	-	-
R1: Die Applikationen muss auf android devices la...	-	-	-

Hinweis: Ein 'X' kennzeichnet eine direkte Verknüpfung im Pflichtenheft bzw. den Anforderungsdokumenten.

3. End-to-End Rückverfolgbarkeits-Baum

Hier ist der vollständige Trace-Pfad von den Anwendungsfällen über die funktionalen Anforderungen bis hin zu den konkreten Code-Implementierungen (z. B. React Native-Dateien, Arduino-Skripte) dargestellt.

Anwendungsfall UC-1: Trainingsgerät verbinden

- **Anforderung FA1:** Das System muss einen Reiter oder eine Navigationsmöglichkeit für Hardwaregeräte / Trainings / vergangene Trainingseinheiten anzeigen. (aus UC-1, UC-2, UC-3)

- **Implementiert in:** `app/app/(tabs)/_layout.tsx` (Zeile 2)

Kontext: * @implements FA1

- **Implementiert in:** `doc/Pflichtenheft/pflichtenheft.tex` (Zeile 217)

Kontext: \item \textbf{FA1:} Das System muss einen Reiter oder eine Navigationsmöglichkeit für Hardwaregeräte / Trainings / vergangene Trainingseinheiten anzeigen. (aus UC-1, UC-2, UC-3)

- **Anforderung FA2:** Das System muss mir eine Liste von verfügbaren Hardwaregeräten und mit welchen Hardwaregeräten ich verbunden bin anzeigen. (aus UC-1)

- **Implementiert in:** `app/app/(tabs)/index.tsx` (Zeile 2)

Kontext: * @implements FA2, FA3, FA4, FA5, FA6

- **Implementiert in:** `app/components/SensorCard.tsx` (Zeile 2)

Kontext: * @implements FA2, FA3, NF3

- **Implementiert in:** `doc/Pflichtenheft/pflichtenheft.tex` (Zeile 218)

Kontext: \item \textbf{FA2:} Das System muss mir eine Liste von verfügbaren Hardwaregeräten und mit welchen Hardwaregeräten ich verbunden bin anzeigen. (aus UC-1)

- **Anforderung FA3:** Das System muss einen Verbindungsaufbau mit dem Hardwaregerät herstellen können. (aus UC-1)

- **Implementiert in:** `app/app/(tabs)/index.tsx` (Zeile 2)

Kontext: * @implements FA2, FA3, FA4, FA5, FA6

- **Implementiert in:** `app/components/SensorCard.tsx` (Zeile 2)

Kontext: * @implements FA2, FA3, NF3

- **Implementiert in:** `app/hooks/useBLE.ts` (Zeile 2)

Kontext: * @implements FA3, FA5, NF2

- **Implementiert in:** `doc/Pflichtenheft/pflichtenheft.tex` (Zeile 219)

Kontext: \item \textbf{FA3:} Das System muss einen Verbindungsaufbau mit dem Hardwaregerät herstellen können. (aus UC-1)

Anwendungsfall UC-2: Echtzeit-Training überwachen

- **Anforderung FA1:** Das System muss einen Reiter oder eine Navigationsmöglichkeit für Hardwaregeräte / Trainings / vergangene Trainingseinheiten anzeigen. (aus UC-1, UC-2, UC-3)

- **Implementiert in:** `app/app/(tabs)/_layout.tsx` (Zeile 2)

Kontext: * @implements FA1

- **Implementiert in:** doc/Pflichtenheft/pflichtenheft.tex (Zeile 217)

Kontext: \item \textbf{FA1:} Das System muss einen Reiter oder eine Navigationsmöglichkeit für Hardwaregeräte / Trainings / vergangene Trainingseinheiten anzeigen. (aus UC-1, UC-2, UC-3)

- **Anforderung FA4:** Das System muss eine Detailansicht für ein ausgewähltes Training anzeigen. (aus UC-2)

- **Implementiert in:** app/app/(tabs)/index.tsx (Zeile 2)

Kontext: * @implements FA2, FA3, FA4, FA5, FA6

- **Implementiert in:** doc/Pflichtenheft/pflichtenheft.tex (Zeile 220)

Kontext: \item \textbf{FA4:} Das System muss eine Detailansicht für ein ausgewähltes Training anzeigen. (aus UC-2)

- **Anforderung FA5:** Das System muss Bewegungsdatenströme empfangen und verarbeiten können. (aus UC-2)

- **Implementiert in:** app/app/(tabs)/index.tsx (Zeile 2)

Kontext: * @implements FA2, FA3, FA4, FA5, FA6

- **Implementiert in:** app/hooks/useBLE.ts (Zeile 2)

Kontext: * @implements FA3, FA5, NF2

- **Implementiert in:** app/hooks/useWebSocket.ts (Zeile 2)

Kontext: * @implements FA5

- **Implementiert in:** app/store/index.ts (Zeile 2)

Kontext: * @implements FA5

- **Implementiert in:** doc/Pflichtenheft/pflichtenheft.tex (Zeile 221)

Kontext: \item \textbf{FA5:} Das System muss Bewegungsdatenströme empfangen und verarbeiten können. (aus UC-2)

- **Implementiert in:** embedded/src/Executable.ino (Zeile 2)

Kontext: * @implements FA5

- **Implementiert in:** embedded/src/Training_Skript/Training_Skript.ino (Zeile 2)

Kontext: // @implements FA5

- **Anforderung FA6:** Das System muss die empfangenen Bewegungen in Echtzeit visualisieren. (aus UC-2)

- **Implementiert in:** app/app/(tabs)/index.tsx (Zeile 2)

Kontext: * @implements FA2, FA3, FA4, FA5, FA6

- **Implementiert in:** app/components/LiveChart.tsx (Zeile 2)

Kontext: * @implements FA6

- **Implementiert in:** doc/Pflichtenheft/pflichtenheft.tex (Zeile 222)

Kontext: \item \textbf{FA6:} Das System muss die empfangenen Bewegungen in Echtzeit visualisieren. (aus UC-2)

Anwendungsfall UC-3: Trainingsdaten einsehen

- **Anforderung FA1:** Das System muss einen Reiter oder eine Navigationsmöglichkeit für Hardwaregeräte / Trainings / vergangene Trainingseinheiten anzeigen. (aus UC-1, UC-2, UC-3)

- **Implementiert in:** app/app/(tabs)/_layout.tsx (Zeile 2)

Kontext: * @implements FA1

- **Implementiert in:** doc/Pflichtenheft/pflichtenheft.tex (Zeile 217)

Kontext: \item \textbf{FA1:} Das System muss einen Reiter oder eine Navigationsmöglichkeit für Hardwaregeräte / Trainings / vergangene Trainingseinheiten anzeigen. (aus UC-1, UC-2, UC-3)

- **Anforderung FA7:** Das System muss historische Bewegungsdaten grafisch anzeigen können. (aus UC-3)

- **Implementiert in:** app/app/(tabs)/history.tsx (Zeile 2)

Kontext: * @implements FA7

- **Implementiert in:** app/components/SessionCard.tsx (Zeile 2)

Kontext: * @implements FA7

- **Implementiert in:** doc/Pflichtenheft/pflichtenheft.tex (Zeile 223)

Kontext: \item \textbf{FA7:} Das System muss historische Bewegungsdaten grafisch anzeigen können. (aus UC-3)